

Module 13) >>2. Programming Style

- **Understanding Python's PEP 8 guidelines.**

What is PEP 8?

PEP stands for "Python Enhancement Proposal." PEP 8 specifically is:

- The style guide for Python code
- Originally written by Guido van Rossum (Python's creator) in 2001
- Regularly updated as Python evolves

Key PEP 8 Guidelines

1. Code Layout & Indentation

- **Indentation:** Use 4 spaces per indentation level (never mix tabs and spaces)
- **Maximum line length:** 79 characters for code, 72 for docstrings/comments
- **Blank lines:**
 - Two blank lines around top-level functions and classes
 - One blank line around method definitions inside classes
- **Imports:**
 - Should be on separate lines
 - Grouped in this order with blank lines between:

2. Naming Conventions

- **Variables:** lowercase_with_underscores (snake_case)
- **Constants:** UPPERCASE_WITH_UNDERSCORES
- **Functions/Methods:** lowercase_with_underscores
- **Classes:** CapitalizedWords (PascalCase)
- **Private:** Single leading underscore `_private_var`

- **Strongly Private:** Double leading underscore `__strongly_private`

3. Whitespace

- **Avoid extraneous whitespace:**
 - Inside parentheses, brackets, or braces
 - Before commas, colons, or semicolons

4. Comments

- Block comments: Start with `#` and one space
- Inline comments: Use sparingly, separated by at least two spaces from code
- Docstrings: Use triple quotes (`"""`) for module, function, class, and method documentation

5. String Quotes

- Use single quotes (`'`) and double quotes (`"`) interchangeably, but be consistent
- For triple-quoted strings, use double quotes (`"""`)

6. Trailing Commas

```
colors = [  
    'red',  
    'blue',  
    'green',  
]
```

- **Indentation, comments, and naming conventions in Python.**

Indentation in Python

Python uses indentation to define code blocks (unlike many languages that use braces). Proper indentation is critical for correct execution.

Comments in Python

1. Single-line Comments
2. Multi-line Comments (Docstrings)

Naming Conventions

1. Variables and Functions
2. Constants

```
MAX_CONNECTIONS = 100
DEFAULT_TIMEOUT = 30
```

3. Classes
4. Private/Protected Members
5. Special Classes

- **Writing readable and maintainable code**

- 1 Fundamental Principles:
 2. Code Organization
- Module Organization

```
import os
import sys
from typing import List, Dict
```

```
import numpy as np
import pandas as pd
```

```
from .utils import helper_function
from .config import SETTINGS
```

3. Writing Clean Functions:

Short (ideally < 20 lines)

Single responsibility

Descriptive names

Few parameters (3-4 max)

No side effects (unless explicitly needed)

1. Effective Naming:

- Best Practices
- Variables: Nouns (customer, total_price)
- Functions: Verbs (calculate_total, validate_input)
- Classes: Nouns (Customer, PaymentProcessor)
- Booleans: Prefix with 'is', 'has', 'can' (is_active, has_permission)

2. Documentation:

3. Error Handling:

4. Testing and Maintainability

5. Tools for maintaining Code Quality